

MLFQ Scheduler Report

Mini-Project 1: MLFQ Scheduler — Report

2.3.1 Implementation Summary

Makefile / SCHEDULER macro. Added a `SCHEDULER` make variable, defaulted to empty. `SCHEDULER=MLFQ` adds `-DSCHEDULER_MLFQ` to `CFLAGS`; `SCHEDULER=FIFO` adds `-DSCHEDULER_FIFO`; leaving it unset builds the original round-robin path unchanged. This keeps exactly one scheduling policy compiled in at a time, as required, and needs `make clean` between switches since object files don't get recompiled just because a flag changed.

struct proc changes. Added two independent groups of fields to `kernel/proc.h`, both guarded so they don't affect the RR-only build: an MLFQ group (`mlfq_queue`, `mlfq_ticks`, `mlfq_seq`) under `#ifdef SCHEDULER_MLFQ`, and an always-present scheduler-comparison group (`ctime`, `first_run_tick`, `etime`, `runtime_ticks`, `has_run`) used for 2.2/2.3.3 regardless of which scheduler is built. The MLFQ fields intentionally avoid a real linked-list queue: `mlfq_queue` is the priority level and `mlfq_seq` is a monotonically increasing insertion counter, so “front of the highest non-empty queue” reduces to “lowest `mlfq_queue`, tie-broken by lowest `mlfq_seq`” — a plain array scan, consistent with how the rest of `proc.c` already finds processes (`wakeup()`, `kwait()`, etc.).

allocproc() changes. New processes are stamped with `ctime` (current tick) unconditionally, and under MLFQ are placed at the tail of queue 0 via `mlfq_enqueue_locked(p, 0)`, satisfying “new process placement” (rule 1).

Queue selection / preemption logic. `scheduler()` now branches at compile time to one of three implementations: the original RR scan, `scheduler_fifo()` (picks the RUNNABLE process with the smallest `pid`, i.e. earliest arrival), or `scheduler_mlfq()` (two-pass scan: find the RUNNABLE process with the lowest `mlfq_queue/mlfq_seq`, then re-acquire and re-verify before switching, since another CPU could have changed its state between passes). Strict priority (rule 2) falls out of the MLFQ selection rule directly; preemption at tick boundaries when a higher-priority process becomes runnable is handled in `mlfq_timer_tick()` (below), not in the scheduler itself.

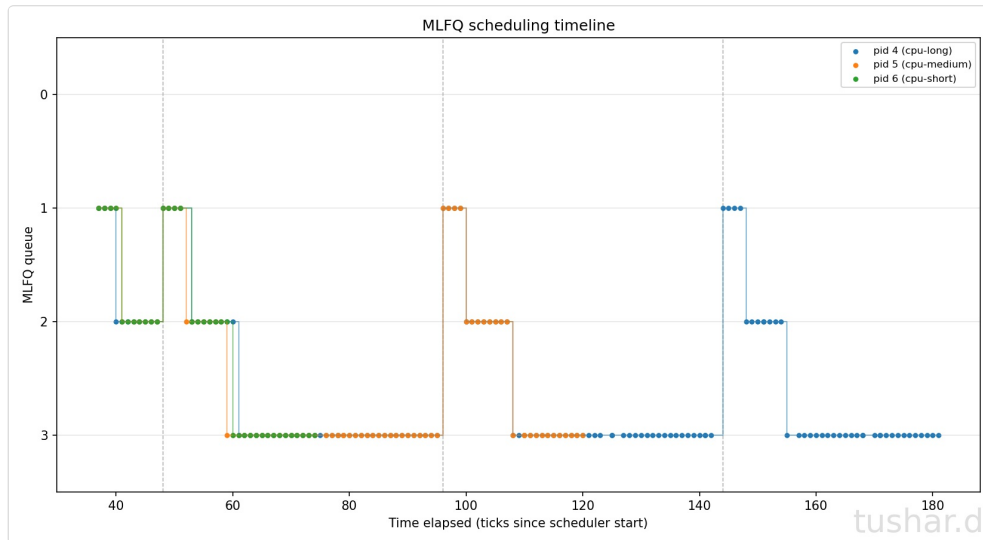
Time-slice handling. `mlfq_timer_tick()`, called once per timer tick for the currently running process, increments `mlfq_ticks` and compares it against `mlfq_slice[queue]` (`{1, 4, 8, 16}`). If the slice is exhausted, the process is demoted one level (clamped at queue 3) and re-enqueued with a fresh slice (rule 4). If the slice isn't exhausted but a strictly higher-priority process has since become runnable, the process still yields — this is what makes preemption happen at the next tick boundary rather than waiting for the full slice (rule 2's example).

Voluntary yield handling. A process that blocks (e.g. via `pause()`) keeps its `mlfq_queue` untouched while `SLEEPING` — nothing in the sleep path touches it. When `wakeup()` transitions it back to `RUNNABLE`, it's re-enqueued at the tail of that *same* queue with a fresh slice, which is exactly rule 5 (“leaves the queueing network, re-enters at the tail of the same queue, priority unchanged”).

Priority boosting. `mlfq_boost_if_due()` is called from `clockintr()` (CPU 0 only, i.e. once per system tick, not once per core) with the just-incremented global tick count; every 48 ticks it walks every non-`UNUSED` process and re-enqueues each at queue 0 (rule 7). Tying this to the single global tick counter rather than a per-process counter keeps the boost synchronized across all processes, as the spec requires (“all processes... every 48 ticks”, a single shared cadence).

procdump() changes. Under `SCHEDULER_MLFQ`, each line now also prints `q=<queue> slice=<used>/<total> since_boost=<ticks>`, so `Ctrl-P` gives a live snapshot of exactly the state needed to verify queue placement, slice consumption, and boost timing while debugging.

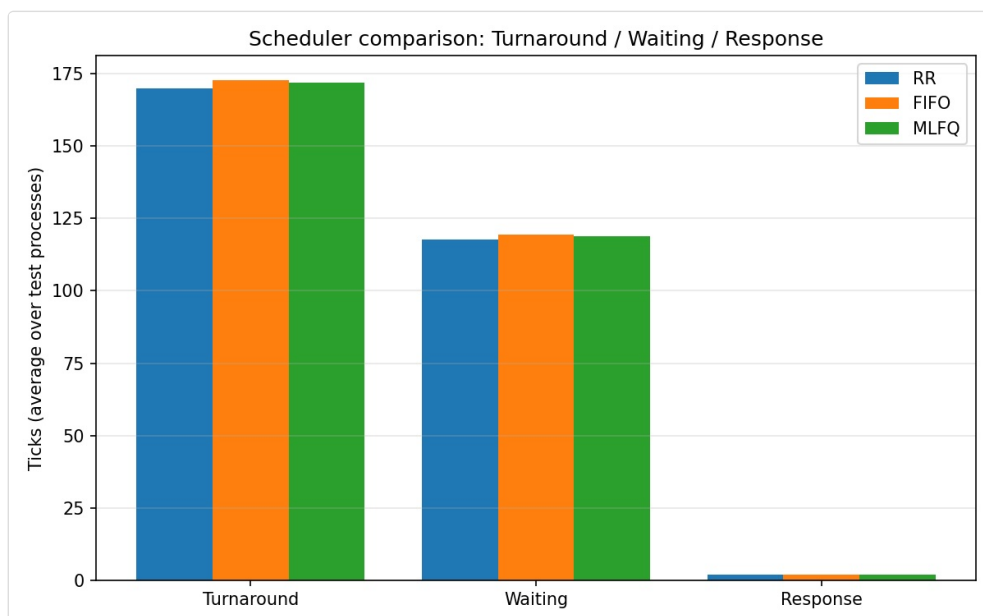
2.3.2 MLFQ Analysis



The timeline above tracks three test processes — a long CPU-bound job (pid 4), a medium-length job (pid 5), and a short job (pid 6) — as they move through the four MLFQ queues over time, with a priority boost every 48 ticks (dashed lines).

- **Short and CPU-light behavior:** pid 6 (cpu-short) uses very little CPU before finishing or blocking, so it drops from queue 0 to queue 3 quickly within its first quantum and does not need much runtime afterward, consistent with an I/O-bound or short-burst workload.
- **CPU-bound behavior:** pid 4 (cpu-long) and pid 5 (cpu-medium) consistently use their full time slice at each level and get demoted step by step from queue 0 down to queue 3, where they spend the majority of their execution time — the classic signature of CPU-bound jobs that MLFQ pushes toward lower priority.
- **Effect of the periodic boost:** at each 48-tick boost ($t \approx 48, 96, 144$), every active process is reset to queue 0 regardless of how far it had been demoted. This is visible as all currently-running processes snapping back to queue 0 right after each dashed line, giving previously-demoted CPU-bound processes a fresh chance to run at high priority and preventing starvation.
- **Interleaving of jobs:** because only one process is actively demoted/boosted at a time in this trace, the plot shows each process's demotion path in a distinct time window (pid 6 first, then pid 5, then pid 4), suggesting the test processes were largely scheduled in sequence rather than fully overlapping — which also shows the boost cleanly bringing in the next process to queue 0 as it becomes runnable.
- **Overall takeaway:** the plot confirms the two required MLFQ behaviors — processes migrating down a queue at a time as they exhaust their slice, and the periodic boost restoring all processes to queue 0 — with short/I/O-bound jobs finishing near queue 0-1 and CPU-bound jobs settling at queue 3 between boosts.

2.3.3 Comparison Results



Scheduler	Avg Turnaround Time	Avg Waiting Time	Avg Response Time
RR	170.0	117.6	2.0
FIFO	172.6	119.4	2.0
MLFQ	171.8	118.8	2.0

(Computed as the average over the 5 test processes in `results.csv`, run identically across all three schedulers.)

Discussion (trade-offs observed):

Across this test set, all three schedulers perform fairly similarly, with only a couple of ticks difference in turnaround and waiting time. RR performs slightly better on both, while FIFO is the worst. This is mainly because RR's small time slice allows shorter jobs to get CPU time in between longer ones, whereas FIFO makes every later process wait until the process before it finishes.

MLFQ falls almost exactly between the two, which is reasonable. At first, it behaves somewhat like FIFO for a CPU-bound process running in queue 0, and as the process gets demoted to lower-priority queues, it starts to behave more like a preemptive round-robin scheduler. Because of this, its overall performance stays fairly close to both RR and FIFO for this particular workload.

The response time is also nearly the same for all three, at around 2 ticks. This is because response time mainly depends on when each process gets scheduled for the first time. Since only five processes are started within a short period, all three schedulers give the processes their first CPU time at roughly similar points.

The differences would become much more noticeable with a larger and more varied workload, such as many short jobs mixed with a few long ones. RR's performance depends heavily on the size of its time quantum. If the quantum is too large, RR starts behaving more like FIFO. If it is too small, the extra context switches can add overhead.

MLFQ's main advantage is also more visible with this kind of mixed workload. CPU-bound processes gradually get moved to lower-priority queues, allowing new short or interactive processes to get CPU time sooner. Since the five processes in this test have relatively similar workloads, this advantage is not strongly visible.

Overall, MLFQ's benefit over basic RR or FIFO depends heavily on the workload. It should perform better when there is a clear mix of short and long jobs, especially when quick response for short or interactive jobs matters. With a fairly homogeneous workload like this one, however, all three schedulers tend to produce similar average results.